

Sistemas Operacionais — Atividade 1 (AV1)

Processos, Threads, Escalonamento e Inferência Local com Ollama

Modalidade: trabalho em equipe, de 5 a 7 discentes

Unidade: AV1

Lançamento: 30/08/2026

Reserva da trilha e do modelo: até 04/09/2026, às 23h59

Entrega do repositório e relatório: 15/09/2026, às 23h59

Apresentação oral: 16/09/2026

1. Contexto

Aplicações de IA generativa executadas localmente não envolvem apenas um modelo de linguagem. Elas mobilizam processos, threads, memória, armazenamento, comunicação em rede, arquivos, logs, bibliotecas, runtimes e mecanismos de proteção do sistema operacional.

Nesta atividade, cada equipe deverá instalar e executar localmente um sistema baseado em **Ollama**, selecionando uma camada de aplicação entre as opções definidas neste documento e escolhendo um modelo de linguagem com **até 10 bilhões de parâmetros**. O modelo deverá ser identificado, justificado e registrado no Google Classroom antes do início dos experimentos.

A atividade tem como objetivo relacionar os conceitos de processos, threads, chamadas de sistema, escalonamento e uso de recursos a uma aplicação real de IA generativa executada em ambiente Linux.

Pergunta norteadora: como a camada de aplicação, o modelo de linguagem, a quantidade de parâmetros, a quantização e a configuração de execução afetam processos, threads, uso de CPU, memória, armazenamento e responsividade de um sistema local de IA generativa?

2. Objetivos de aprendizagem

Ao concluir a atividade, a equipe deverá ser capaz de:

1. Instalar, configurar e executar uma aplicação local de IA generativa baseada em Ollama.
2. Selecionar e justificar o uso de um modelo de linguagem de até 10 bilhões de parâmetros disponível no Hugging Face e compatível com execução local.
3. Identificar processos, threads, estados, hierarquia e chamadas de sistema associados à execução da aplicação.
4. Observar e comparar o consumo de CPU, memória, armazenamento e, se disponível, GPU em diferentes configurações.
5. Relacionar concorrência, escalonamento, filas de requisições e utilização de recursos ao funcionamento de aplicações de IA generativa.
6. Avaliar criticamente sugestões de código, configuração ou explicação produzidas por IA generativa, confrontando-as com documentação e evidências observadas no sistema.

3. Organização geral da atividade

Todas as equipes deverão utilizar:

- **Ollama** como runtime obrigatório para execução local do modelo;
- um ambiente Linux, instalação nativa, máquina virtual, WSL ou contêiner, devidamente documentado;
- um modelo de linguagem de até 10 bilhões de parâmetros;
- uma das camadas de aplicação previstas na Seção 4;
- uma aplicação instalada, executável e observável no ambiente da equipe.

O Ollama deverá ser executado localmente e deverá ser utilizado para carregar e atender requisições ao modelo selecionado. A camada de aplicação poderá se comunicar com o Ollama por meio da API local.

4. Camadas de aplicação disponíveis

Cada equipe deverá selecionar **uma** das trilhas abaixo. A escolha deverá ser registrada no Google Classroom junto com o modelo selecionado.

Trilha A — Chat local: Ollama + Open WebUI

Camada de aplicação: Open WebUI

Runtime obrigatório: Ollama

O Open WebUI é uma plataforma auto-hospedada, extensível e capaz de operar offline, com integração a runtimes locais como Ollama. Permite estudar a interação entre uma interface web, serviços locais, API HTTP, contêineres, portas, volumes, persistência e execução de modelos.

Foco de observação:

- processos do Ollama e do Open WebUI;
- comunicação HTTP pela API local;
- portas, sockets e conexões;
- execução nativa ou em contêiner;
- volumes, diretórios persistentes e logs;
- consumo de memória durante o carregamento e uso do modelo;
- comportamento com múltiplas conversas ou requisições.

A equipe deverá documentar se o Ollama será executado no host, em contêiner próprio ou em contêiner integrado ao Open WebUI; deverá registrar portas, variáveis de ambiente, volumes e mecanismo de comunicação entre os componentes. A documentação do Open WebUI orienta, para Ollama executado no host, o uso de `http://host.docker.internal:11434` como endereço do provedor no contêiner. A documentação também oferece caminhos de instalação com Docker e volumes persistentes, tanto para CPU quanto para GPU.

Trilha B — RAG textual: Ollama + ollama-local-rag

Camada de aplicação: [cpepper96/ollama-local-rag](https://github.com/cpepper96/ollama-local-rag)

Runtime obrigatório: [Ollama](https://ollama.com)

O repositório apresenta uma aplicação RAG simples com LangChain, Ollama e FAISS, utilizando documentos locais.

Foco de observação:

- processos Python, Ollama e bibliotecas auxiliares;
- leitura e indexação de documentos;
- criação e persistência de índices vetoriais;
- consumo de CPU e memória nas fases de indexação e consulta;
- tamanho e acesso aos arquivos de documentos e índices;
- comparação entre perguntas curtas, longas e consultas com recuperação de contexto.

Trilha C — RAG sobre PDF: Ollama + ollama_pdf_rag

Camada de aplicação: [tonykipkemboi/ollama_pdf_rag](https://github.com/tonykipkemboi/ollama_pdf_rag)

Runtime obrigatório: [Ollama](https://ollama.com)

O repositório apresenta uma aplicação local para consulta a arquivos PDF utilizando Ollama, LangChain e componentes de interface ou notebook. [web:109]

Foco de observação:

- leitura e extração de dados de arquivos PDF;
- consumo de armazenamento, CPU e memória durante chunking e indexação;
- processos da aplicação, do Ollama e da interface;
- diferença entre carga de indexação e carga de inferência;
- efeitos do tamanho e da quantidade de documentos;
- logs, dados temporários e persistência de artefatos.

5. Escolha e reserva do modelo

5.1 Regra de exclusividade

Cada equipe deverá registrar, na thread específica do Google Classroom, a trilha e o modelo escolhidos. O registro deverá conter o nome da equipe, os integrantes, o nome completo do modelo, o URL do model card, a família, o checkpoint/variante, o número de parâmetros, o formato, a quantização, a licença e uma justificativa de até 100 palavras.

A prioridade será definida pela ordem cronológica de registros completos e válidos. Nenhuma outra equipe poderá utilizar o mesmo modelo-base, entendido como a mesma combinação de família, checkpoint/variante e faixa de parâmetros, ainda que esteja em outro formato ou nível de quantização.

Cada equipe deverá escolher um modelo de linguagem com **até 10 bilhões de parâmetros** e registrá-lo na thread específica criada no **Google Classroom**.

O registro deverá incluir:

1. nome da equipe;
2. integrantes;
3. trilha de aplicação escolhida;
4. nome completo do modelo;
5. URL da página do modelo no Hugging Face;
6. número de parâmetros;
7. variante, formato e quantização que serão utilizados;
8. licença;
9. justificativa curta da escolha;
10. data e horário do registro.

A primeira equipe que registrar corretamente um modelo na thread terá prioridade sobre ele. A partir desse registro, as demais equipes **não poderão selecionar o mesmo modelo-base**, ainda que pretendam utilizar quantização, formato ou variante diferente. A regra busca assegurar diversidade de modelos e comparações mais ricas entre as equipes. Considera-se “mesmo modelo-base” aquele que compartilha a mesma arquitetura, família, tamanho de parâmetros e checkpoint principal de origem, ainda que seja disponibilizado em outro formato, quantização ou repositório de conversão.

É importante diferenciar:

- **modelo-base:** por exemplo, Qwen, Mistral, Gemma, Phi ou Falcon;
- **checkpoint/variante:** por exemplo, Base, Instruct, Chat ou Coder;
- **formato:** Safetensors, GGUF ou outro;
- **quantização:** Q4, Q5, Q8, FP16 etc.

A escolha somente será considerada válida após a confirmação do docente ou após o prazo indicado para impugnações técnicas. Caso o modelo não seja compatível com Ollama, exceda o limite de parâmetros, não tenha licença adequada ou não possa ser executado no ambiente da equipe, a escolha poderá ser invalidada e a equipe deverá selecionar outro modelo disponível.

Plano de contingência: caso o modelo inicialmente selecionado não seja executável no ambiente da equipe, esta deverá registrar a evidência do impedimento — erro, consumo insuficiente de memória, incompatibilidade de instruções da CPU, ausência de espaço em disco ou indisponibilidade de GPU — e migrar para outro modelo elegível de menor porte, preservando a trilha e documentando a alteração.

Perfil de hardware	Faixa de modelo recomendada
CPU e 8 GB de RAM	1B a 3B; preferencialmente em quantização leve
CPU e 16 GB de RAM	3B a 7B, geralmente quantizado
16 GB ou mais + GPU	7B a 10B, conforme a VRAM e a quantização

Perfil de hardware	Faixa de modelo recomendada
GPU com VRAM limitada	Testar modelo quantizado e registrar limite de VRAM

5.2 Limite de parâmetros

O modelo-base selecionado deve possuir **até 10 bilhões de parâmetros**, conforme informado pelo model card, pela documentação oficial do produtor ou por fonte técnica confiável. O tamanho do arquivo baixado não substitui a informação sobre a quantidade de parâmetros.

Não são aceitos:

- modelos com mais de 10 bilhões de parâmetros, mesmo que fortemente quantizados;
- modelos cuja quantidade de parâmetros não possa ser confirmada;
- modelos exclusivamente remotos ou dependentes de API paga;
- modelos sem condições de uso claramente identificadas;
- variantes que não possam ser executadas localmente com Ollama ou com a camada escolhida.

5.3 Critérios obrigatórios para seleção no Hugging Face

A equipe deverá analisar e justificar os seguintes critérios no relatório.

Critério	O que verificar	Relevância para Sistemas Operacionais
Identificação	Organização, nome completo e URL do model card	Rastreabilidade e reprodutibilidade
Parâmetros	Quantidade total de parâmetros do modelo-base	Estimativa de RAM/VRAM e custo computacional

Finalidade	Base, instruct, chat, código, multilíngue ou outra	Adequação à camada de aplicação e aos testes
Idioma	Suporte ao português e demais idiomas necessários	Qualidade das entradas e saídas avaliadas
Licença	Permissões, restrições e obrigações de uso	Conformidade e uso responsável
Formato	GGUF, Safetensors, GGML ou outro	Compatibilidade com runtime e armazenamento
Quantização	Ex.: Q4, Q5, Q8, FP16, quando aplicável	Compromisso entre memória, velocidade e qualidade
Contexto	Janela máxima de contexto e limitações	Memória, cache e carga de trabalho
Tamanho dos arquivos	Dimensão dos arquivos baixados	Espaço em disco, tempo de download e E/S
Requisitos de hardware	RAM, VRAM, CPU/GPU e instruções recomendadas	Viabilidade no ambiente local
Compatibilidade	Disponibilidade para Ollama ou conversão compatível	Execução local obrigatória
Documentação	Clareza do model card, exemplos e limitações	Reprodutibilidade e análise crítica
Riscos e limitações	Viés, confabulação, segurança, privacidade, limites conhecidos	Interpretação responsável dos resultados

O Hugging Face permite explorar modelos com filtros de parâmetros e de bibliotecas/runtimes, inclusive llama.cpp, MLX e Ollama.

5.4 Exemplos de famílias elegíveis

As equipes deverão verificar a variante exata antes de registrá-la. Famílias potencialmente elegíveis incluem modelos de 1B, 2B, 3B, 4B, 7B, 8B, 9B e 10B parâmetros, desde que sejam compatíveis com execução local e atendam aos critérios desta atividade.

Exemplos que podem ser investigados:

- Mistral 7B: o model card do Mistral-7B-v0.1 informa 7 bilhões de parâmetros.
- Yi-Coder 9B Chat: modelo voltado a código, identificado como 9B no Hugging Face.
- Falcon3-10B-Instruct: modelo instruct de 10 bilhões de parâmetros; o card informa suporte a múltiplos idiomas, incluindo português.
- Qwen, Gemma, Phi, SmolLM e outras famílias com variantes de até 10B, desde que a equipe confirme os requisitos e a compatibilidade da variante escolhida.

Os exemplos não constituem lista fechada e não garantem disponibilidade, licença, desempenho ou compatibilidade em todos os ambientes.

6. Parte A — Instalação e documentação do ambiente

6.1 Inventário do ambiente

Registre:

- distribuição e versão do sistema operacional;
- versão do kernel;
- processador e número de núcleos/threads;
- memória RAM disponível;
- GPU e VRAM, se houver;
- armazenamento disponível;
- versão do Ollama;
- versão da linguagem e dependências da camada de aplicação;
- ambiente de execução: nativo, máquina virtual, WSL ou contêiner.

Comandos sugeridos:

```
uname -a  
cat /etc/os-release  
lscpu
```

```
free -h  
nproc  
df -h  
lsblk  
ollama --version  
ollama list
```

Se houver GPU NVIDIA, registrar também:

```
nvidia-smi
```

6.2 Ficha da camada de aplicação

Documente:

- nome e URL do repositório GitHub;
- autor ou organização;
- licença;
- commit, tag ou release utilizado;
- data de acesso;
- finalidade da aplicação;
- arquitetura simplificada;
- dependências;
- comandos de instalação e execução;
- adaptações realizadas;
- limitações e erros encontrados.

6.3 Instalação e execução inicial

Instale e execute Ollama, a camada de aplicação escolhida e o modelo reservado. Registre:

- comandos utilizados;
- tempo de download do modelo, se realizado durante a atividade;

- espaço ocupado pelos arquivos;
- tempo de inicialização;
- processos criados;
- portas e serviços utilizados;
- logs, erros e avisos relevantes.

7. Parte B — Observação de processos, threads e chamadas de sistema

7.1 Processos e threads

Com a aplicação em execução, identifique:

- processo principal e processos auxiliares;
- PID e PPID;
- estados dos processos;
- quantidade de threads;
- consumo de CPU e memória;
- relação entre Ollama, camada de aplicação e demais componentes;
- possíveis trabalhadores de inferência, indexação, interface e recuperação de documentos.

Comandos sugeridos:

```
ps -eo pid,ppid,stat,ni,pri,psr,pcpu,pmem,nlwp,comm --sort=-pcpu
ps -eLf
pstree -p
htop
top -H
```

7.2 Chamadas de sistema

Execute uma sessão curta da aplicação ou do runtime com `strace`:

```
strace -f -c -o strace-resumo.txt seu_comando
```

Quando o serviço já estiver em execução, a equipe poderá anexar o `strace` ao PID, se tiver permissões:

```
strace -f -c -p PID_DO_PROCESSO -o strace-resumo.txt
```

Analise pelo menos três chamadas ou famílias de chamadas e relacione-as a:

- criação, execução ou espera de processos;
- leitura e gravação de arquivos;
- carregamento de pesos ou dados;
- alocação ou mapeamento de memória;
- rede e comunicação local;
- criação de logs;
- acesso a diretórios, índices ou documentos.

Não é necessário interpretar todas as chamadas listadas. A equipe deverá explicar por que as chamadas selecionadas são relevantes para a execução do sistema.

8. Parte C — Experimentos comparativos

Os experimentos deverão utilizar o mesmo modelo e a mesma camada de aplicação, salvo quando a comparação envolver uma variação de quantização permitida pela documentação do próprio modelo. Todas as configurações devem ser descritas de forma suficiente para reprodução.

Configuração 1 — Execução padrão

Executar o sistema segundo as instruções do repositório e da documentação do modelo, sem alterações significativas.

Configuração 2 — Concorrência ou carga controlada

Alterar, quando tecnicamente possível:

- número de requisições simultâneas;
- número de threads ou trabalhadores;
- tamanho da fila;
- tamanho da entrada ou do contexto;
- quantidade de documentos indexados;
- número de conversas paralelas;
- outro parâmetro justificável da camada de aplicação ou do runtime.

A equipe deverá explicar a alteração, a hipótese de impacto e o método adotado.

Configuração 3 — Ajuste de execução local

Comparar uma segunda condição técnica, como:

- quantização diferente do mesmo modelo, quando disponível e permitida;
- contexto curto versus contexto longo;
- cache habilitado versus desabilitado;
- execução em CPU versus GPU, quando disponível;
- execução nativa versus em contêiner;
- índices ou datasets pequenos versus maiores, nas trilhas RAG;
- outro ajuste aprovado pelo docente.

A equipe não deverá utilizar outro modelo-base na Configuração 3, pois a exclusividade dos modelos é parte do desenho comparativo entre equipes.

8.1 Repetições e carga de teste

Realize, preferencialmente:

- dois tamanhos de entrada, contexto, lote ou conjunto de documentos;
- duas condições de concorrência ou carga;
- ao menos três repetições por configuração.

A equipe poderá adaptar os valores à infraestrutura disponível, desde que documente e justifique as escolhas.

8.2 Métricas mínimas

Registre, para cada configuração:

- tempo de inicialização;
- tempo de carregamento do modelo, quando mensurável;
- tempo até a primeira resposta ou token, se disponível;
- tempo total da requisição;
- tokens por segundo, se disponível;
- tempo médio por requisição;
- CPU utilizada;
- memória RAM utilizada;
- VRAM utilizada, se aplicável;
- quantidade de processos e threads;
- espaço ocupado por modelo, índices e dados;
- ocorrência de erros, bloqueios, falhas de log ou resultados inconsistentes;
- critério simples e explícito de avaliação da resposta gerada.

Apresente os resultados em tabelas e inclua, no mínimo, um gráfico ou visualização equivalente.

Cada equipe deverá executar pelo menos 12 execuções mensuráveis, cobrindo três configurações — padrão, concorrência/carga modificada e ajuste de execução local —, ao menos

dois tamanhos de entrada ou carga e pelo menos duas repetições por cenário. A equipe deverá registrar parâmetros, comandos, horário, versão do runtime e versão do modelo.

É proibido publicar no repositório chaves, senhas, tokens, arquivos .env, dados pessoais, documentos sigilosos ou pesos dos modelos. Os documentos utilizados nas trilhas RAG deverão ser públicos, sintéticos ou anonimizados. Arquivos grandes e artefatos gerados devem ser excluídos por meio de .gitignore, com instruções no README para sua obtenção ou reprodução.

9. Questões de análise

O relatório deverá responder, com base em evidências produzidas pela equipe:

1. Qual trilha de aplicação foi selecionada e por que ela é apropriada para a disciplina?
2. Qual modelo foi selecionado, quais são seus parâmetros e por que ele foi escolhido?
3. Como formato, quantização, contexto e tamanho do modelo afetaram os requisitos de armazenamento, RAM ou VRAM?
4. Quais processos e threads foram observados durante a inicialização e a inferência?
5. Qual é a relação de parentesco e responsabilidade entre os processos identificados?
6. Quais chamadas de sistema foram relevantes para a carga do modelo, para a leitura de dados, para a comunicação ou para a geração de logs?
7. O aumento da concorrência melhorou a responsividade ou a vazão? Quais custos apareceram?
8. Houve sinais de competição por CPU, memória, armazenamento, GPU, rede ou outro recurso?
9. Como entradas ou contextos maiores afetaram o desempenho e o uso de recursos?
10. Como a arquitetura local contribui para privacidade, disponibilidade, autonomia e controle de dados? Quais riscos permanecem?
11. Como o sistema poderia se comportar diante de múltiplos usuários ou requisições simultâneas?

12. Quais limitações do hardware, do runtime, da camada de aplicação e do modelo afetam a validade dos resultados?
13. Quais informações geradas por IA precisaram ser verificadas, corrigidas ou rejeitadas?

Conceito	Definição operacional sugerida
Tempo de inicialização	Tempo entre o comando de execução e a aplicação estar pronta para receber uma requisição
Latência	Tempo total entre enviar a requisição e receber a resposta completa
TTFT	Tempo até a primeira resposta ou primeiro token, quando a aplicação disponibilizar esse dado
Vazão	Número de requisições concluídas ou tokens produzidos por unidade de tempo
Uso de CPU	Percentual de CPU observado durante a execução
Memória	Pico ou média de RAM utilizada durante a carga
VRAM	Memória de GPU ocupada, quando houver GPU disponível
Taxa de erro	Número de falhas, timeouts, travamentos ou respostas inválidas por número de tentativas

10. Uso crítico de IA generativa

Ferramentas de IA generativa podem apoiar instalação, leitura de código, produção de scripts, elaboração de hipóteses e documentação. Porém, deverão ser utilizadas de maneira transparente e verificável.

Cada equipe deverá anexar uma **Declaração de Uso de IA Generativa** com:

- ferramenta e modelo utilizados;
- finalidade de cada uso;

- até cinco prompts relevantes;
- síntese das sugestões aproveitadas;
- sugestões corrigidas ou rejeitadas;
- erros encontrados;
- testes, documentação ou observações usadas para verificar as respostas;
- distribuição das contribuições dos integrantes.

Respostas de IA não substituem comandos, logs, dados, medições, código executável, documentação oficial ou análise da equipe.

11. Entregáveis

A equipe deverá disponibilizar em um repositório público do Github cujo nome deve seguir a regra (SO_UFS_2026_2_sobrenome_nome1_sobrenome_nome2, basta nome e sobrenome de dois componentes da equipe):

1. **Relatório técnico em PDF que também deverá ser disponibilizado na área da atividade no Google Classroom da turma;**
2. URL, commit, tag ou release da camada de aplicação selecionada;
3. ficha técnica do modelo Hugging Face;
4. cópia do registro realizado no Google Classroom;
5. README.md com instalação, execução e reprodução dos experimentos;
6. arquivos de configuração, docker-compose, Dockerfile ou scripts utilizados, se houver;
7. scripts de execução e medição;
8. logs, tabelas, dados e gráficos;
9. evidências de processos, threads, chamadas de sistema, memória e armazenamento;
10. Declaração de Uso de IA Generativa;
11. apresentação em PDF ou slides.

O relatório deverá conter as seções:

1. Tabela indicativa do que cada componente da equipe fez na atividade;
2. Introdução e problema;
3. Trilha e camada de aplicação selecionadas;
4. Modelo Hugging Face selecionado;
5. URL do projeto público criado no Github
6. Ambiente experimental;
7. Arquitetura da aplicação;
8. Processos, threads e chamadas de sistema;
9. Configurações e metodologia;
10. Resultados;
11. Discussão;
12. Relação com Sistemas Operacionais e IA generativa;
13. Limitações e ameaças à validade;
14. Declaração de uso de IA generativa;
15. Conclusão;
16. Referências.

12. Apresentação

Cada equipe terá de **10 a 12 minutos** para apresentação e de **3 a 5 minutos** para perguntas.

A apresentação deverá incluir:

- a trilha e a camada de aplicação escolhidas;
- o modelo selecionado no Hugging Face e sua justificativa;
- arquitetura do sistema;

- ambiente e recursos utilizados;
- demonstração breve da aplicação;
- processos, threads e chamadas de sistema observados;
- resultados e gráfico principal;
- comparação entre configurações;
- limitações e riscos;
- relação com execução local de IA generativa;
- uso crítico e verificado de IA generativa durante a atividade.

Todos os integrantes deverão participar da apresentação oral e do vídeo da atividade.

Observações

- A ausência de código, comandos, logs ou instruções suficientes para reprodução poderá limitar a nota máxima da atividade.
- Resultados sem dados, tabelas, gráficos, logs ou evidências suficientes não serão considerados análise experimental completa.
- O uso de código, texto ou configuração de terceiros, inclusive de IA generativa, deverá ser atribuído e documentado.
- A apresentação deverá demonstrar domínio sobre os artefatos entregues; respostas genéricas ou incompatíveis com o material apresentado poderão reduzir a nota da atividade.
- O não registro do modelo na thread do Google Classroom, ou a escolha de modelo já reservado por outra equipe não serão aceitos.

14. Roteiro de entrega e marcos

Data	Marco
------	-------

30/08/2026	Lançamento da atividade; formação ou confirmação das equipes; seleção preliminar da trilha e do modelo
até 04/09/2026, às 23h59	Registro do modelo e da trilha na thread do Google Classroom
até 15/09/2026, às 23h59	Entrega do repositório no projeto público criado pela equipe no Github e relatório em formato pdf no Google Classroom
16/09/2026	Apresentação da Atividade 1 e fechamento da AV1

15. Referências de partida

- SILBERSCHATZ, Abraham; GALVIN, Peter B.; GAGNE, Greg. *Operating System Concepts*. 10. ed. Hoboken: Wiley, 2018.
- TANENBAUM, Andrew S.; BOS, Herbert. *Modern Operating Systems*. 5. ed. Boston: Pearson, 2023.
- KERRISK, Michael. *The Linux Programming Interface*. San Francisco: No Starch Press, 2010.
- LINUX MAN-PAGES PROJECT. *fork(2), execve(2), wait(2), clone(2), pthreads(7) e sched(7)*. Disponível em: <https://man7.org/linux/man-pages/>.
- THE LINUX KERNEL DOCUMENTATION. *Scheduler documentation*. Disponível em: <https://docs.kernel.org/scheduler/>.
- OLLAMA. Repositório oficial. Disponível em: <https://github.com/ollama/ollama>. [web:104]
- OPEN WEBUI. Repositório oficial. Disponível em: <https://github.com/open-webui/open-webui>.
- HUGGING FACE. *Models*. Disponível em: <https://huggingface.co/models>. Acesso em: 26 ago. 2026.

- NIST. *Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile*. NIST AI 600-1, 2024. Disponível em: <https://doi.org/10.6028/NIST.AI.600-1>.

16. Checklist final da equipe

- ☐ Equipe composta por 5 a 7 integrantes.
- ☐ Ollama instalado e executado localmente.
- ☐ Uma trilha de aplicação selecionada.
- ☐ Camada de aplicação instalada e funcional.
- ☐ Modelo de até 10B parâmetros selecionado no Hugging Face.
- ☐ Modelo registrado corretamente na thread do Google Classroom.
- ☐ Modelo ainda não reservado por outra equipe.
- ☐ Model card, licença, parâmetros, formato e quantização documentados.
- ☐ Ambiente experimental documentado.
- ☐ Processos e threads observados com evidências.
- ☐ Chamadas de sistema analisadas.
- ☐ Pelo menos três configurações comparadas.
- ☐ Experimentos repetidos e resultados tabulados.
- ☐ Gráfico ou visualização incluída.
- ☐ Relação explícita com Sistemas Operacionais e IA generativa.
- ☐ Código, logs, scripts e README organizados.
- ☐ Uso de IA generativa declarado e criticamente avaliado.
- ☐ Todos os integrantes preparados para a apresentação.

13. Vídeo obrigatório da equipe

Cada equipe deverá produzir um vídeo de apresentação e demonstração da atividade, com duração máxima de **10 minutos**. Todos os integrantes da equipe deverão participar obrigatoriamente do vídeo, com identificação clara de suas contribuições.

O vídeo não deverá ser anexado ao Google Classroom. A equipe deverá disponibilizar somente uma URL pública ou acessível por link, em plataforma de hospedagem de vídeo ou serviço equivalente que permita reprodução sem solicitação de acesso ao docente.

A URL de acesso ao vídeo deverá constar obrigatoriamente nos seguintes locais:

no README.md do repositório público no GitHub, em seção intitulada “Vídeo da atividade”;

no relatório técnico em PDF, na capa ou em seção inicial de identificação da equipe;

em arquivo específico VIDEO.md ou video.txt no repositório, contendo a URL, a data de gravação e a identificação dos participantes; e

na área da atividade no Google Classroom, exclusivamente como link, quando solicitado pelo docente.

O vídeo deverá apresentar, de modo objetivo:

a identificação da equipe, da trilha e do modelo selecionado;

a demonstração da execução local da aplicação com Ollama;

o ambiente de execução e os recursos computacionais utilizados;

a arquitetura geral, incluindo Ollama e a camada de aplicação;

evidências de processos, threads, chamadas de sistema ou monitoramento de recursos;

os resultados experimentais e o gráfico principal;

a análise dos principais limites, riscos e decisões adotadas; e

a participação de todos os integrantes.

A URL deverá permanecer acessível até o encerramento da avaliação da disciplina. Links indisponíveis, privados sem autorização de acesso ou que não permitam reprodução poderão comprometer a comprovação da entrega.

13.1 Complemento aos entregáveis

Além dos entregáveis já previstos na Seção 11, a equipe deverá incluir:

URL funcional do vídeo de até 10 minutos;

arquivo VIDEO.md ou video.txt no repositório, com a URL e a identificação de todos os participantes;

menção à URL do vídeo no README.md e no relatório técnico em PDF; e

comprovação de participação de todos os integrantes no vídeo.

13.2 Ajuste à apresentação e à avaliação

Todos os integrantes deverão participar da apresentação oral e do vídeo da atividade.

A avaliação da apresentação oral e da participação da equipe considerará também a clareza, a objetividade e a demonstração de domínio técnico no vídeo. O vídeo não substitui a apresentação oral prevista para 16/09/2026.

16.1 Itens adicionais do checklist

☐ Vídeo de até 10 minutos produzido, com participação de todos os integrantes;

☐ URL do vídeo funcional e acessível;

☐ URL registrada no README.md, no relatório em PDF e no arquivo VIDEO.md ou video.txt;

☐ Link do vídeo informado no Google Classroom, quando solicitado pelo docente.